

Computer Science E214

Tutorial 1

1 The Argument Clinic

Write a program `argumentclinic.py` that takes two strings as command-line parameters, which are the names of the two souls who will engage in an argument. Your program should behave in the following way:

```
$ python argumentclinic.py Palin Cleese
Palin is having an argument with Cleese.
```

Thus the first soul specified should be arguing with the second.

Note that simply using the “Run” button in an IDE is unlikely to work, since your program should be expecting command line arguments which are not provided when you click on the button. Instead, try typing the `python argumentclinic.py Palin Cleese` command into the console, and it should execute correctly. Modify your program until it works correctly for this example.

Note that, if you wish to use names and surnames, the following should work (given that your program is correct).

```
$ python argumentclinic.py "Michael Palin" "John Cleese"
Michael Palin is having an argument with John Cleese.
```

Without the quotes, this might happen:

```
$ python argumentclinic.py Michael Palin John Cleese
Michael is having an argument with Palin.
```

This is because the command shell (the program providing you with the command prompt and interpreting your commands) has no way of knowing that the space between a name and surname is any different from the spaces between names of the argumentative individuals. Using the quotes tells the shell that everything inside should be treated as one command-line argument.

If you haven’t made special provision yet, your program will do something like the following when only one name is given:

```
$ python argumentclinic.py Palin
Traceback (most recent call last):
  File "argumentclinic.py", line NNN, in <module>
    stdio.writeln(sys.argv[1] + " is having an argument with " + sys.argv[2] + ".")
IndexError: list index out of range
```

This is because there is only one command-line argument being provided to your program, but your program probably uses `sys.argv[2]`. Note that this error message specifies the filename of the file containing the error, as well as the actual line number and line where the error occurred, and a description of the error type. While these messages can sometimes be misleading, it is extremely valuable to develop the habit of carefully reading your error messages,

and to develop your skill at interpreting them: they are your most effective weapon for efficient debugging of your program.

You can ask `sys.argv` how many arguments it contains using the built-in `len` function via `len(sys.argv)`, which returns the length of the arguments list as an integer. This value is one more than the number of arguments because the python program name is the first entry in the arguments list—that is how Python’s error message was able to include that information in the error message above. Use the number of arguments, together with `if` statements, to change your program to behave in the following way.

```
$ python argumentclinic.py
*crickets*
$ python argumentclinic.py Palin
Palin is arguing with themselves.
$ python argumentclinic.py Palin Cleese
Palin is having an argument with Cleese.
$ python argumentclinic.py Palin Cleese Idle
Much hilarity ensues.
$ python argumentclinic.py Palin Cleese Idle Chapman
Much hilarity ensues.
```

Now, obtain the file `argumentclinic.py` made available with the tutorial, and place it in a new directory. (This is to ensure you do not overwrite your own solution!) Verify that running that version in its directory behaves exactly as specified above. Next use this version to run `python argumentclinic.py Idle Gilliam`. Is the output what you would expect? Study the program, and ensure you understand why the program behaves as it does.

Conclusion: getting the right output for the examples provided does not necessarily mean your program is working as intended. As a result, while we may provide test cases for programming tasks in this course, we will typically hold in reserve unseen test cases to ensure your program generalizes appropriately beyond provided test cases. Therefore, you should test your programs yourself and go about it intelligently — see if you can identify edge cases and possible problem areas, and test your program with your own inputs that target those weaknesses.

2 JBook 1.2.18 (modified)

Write a program `sidelength.py` that takes two floating point numbers as command-line arguments, which are the x - and y -coordinates of a point in the Cartesian plane. The program must then print out the distance from the origin to the point. Example usage:

```
$ python sidelength.py 3.0 0.0
3.0

$ python sidelength.py -3.0 4.0
5.0
```

3 JBook 1.2.30

Write a program `uniformrandomnumbers.py` that prints five uniform random values between 0 and 1, as well as their average value, their minimum value and maximum value. Use the `random()` function from the `random` library, as well as the built-in `min()` and `max()` functions.

Example usage:

```
$ python uniformrandomnumbers.py
0.452614024489044
0.8344356747555637
0.9562642060772457
0.23232342871802303
0.8958481508879242
Minimum: 0.23232342871802303
Maximum: 0.9562642060772457
Average: 0.6742970969855602
```

4 Example: Estimating Probabilities and Averages

This example first aims to give you some guidance around estimating probabilities from simulations. The question is:

Ford Prefect throws a single six-sided die. Estimate by simulation the probability that the top number will be either a 2 or a 4.

The answer is fairly obvious for this question, but you will answer this in an “overkill” fashion which generalises nicely to more complex simulation problems.

Roughly speaking, there are three things we need to define when analysing a problem statistically. They are *the experiment*, the possible *outcomes* of the experiment, and the *event* of interest.

Experiment: The thing of interest that has uncertain outcomes. In this case, it is throwing a six-sided die.

Outcomes: The possible results of running the experiment. In this case, the outcomes are 1, 2, 3, 4, 5 or 6. Each outcome has some probability of occurring. Often this probability is not *explicitly* known; however, in this case, each of the outcomes has a 1/6 chance of happening (assuming the die is fair).

Event: A set of outcomes of interest. In this case, this is the set of outcomes {2, 4}.

In order to **estimate** the probability of an event, we first collect simulation data: we run the experiment N times, record the outcome each time, and count how many times (N_E) the outcome corresponds with the event of interest E . Below is an example of running a series of experiments, recording their outcomes, and checking whether the outcome corresponds to the event of interest.

Experiment Number	Outcome	Event happened? (rolled 2 or 4)
1	4	Yes
2	1	No
3	5	No
4	3	No
5	6	No
6	2	Yes
7	4	Yes

To estimate the probability of the event, we divide the number of times the event happened by the number of experiments. For the data collected above, we have:

$$P(\text{number is 2 or 4}) \approx \frac{\text{Number of times event occurred}}{\text{Number of experiments}} = \frac{N_E}{N} = \frac{3}{7}$$

The correct probability is, of course, $2/6$, so this estimate is not too far off. We can improve the estimate by increasing the number of experiments.

We can do the same in Python using something along the lines of:

```
N = ... number of simulations ...
N_events = 0; # Number of times event occurs

for i in range(N):
    ... Do the experiment ...
    if (outcome corresponds with event):
        N_events += 1

prob = N_events / N
```

We can thus answer the question as follows:

```
import random, stdio

N = 10000
N_events = 0;

for i in range(N):
    die = 1 + int(random.random() * 6)
    if die == 2 or die == 4:
        N_events += 1

prob = N_events / N
stdio.writeln(prob)
```

Put the above code into a Python program file to test it out. This should give you about 0.33 (or $2/6$).

In subsequent questions, the experiments you perform will be more complex, with the outcomes generated and events tested for being more complicated, but the principle is the same: run N simulations, check how many outcomes correspond with the event you are interested in, and take the ratio of the number of times the event occurred to the number of experiments.

Averages: A similar argument to the above can be used for calculating the expected value (average) of a quantity from simulations: run N simulations, total up the value of the quantity in each simulation, and divide the total by the number of repetitions.¹ An example question that you might tackle of this nature:

Ford Prefect repeatedly throws a single six-sided die until the top number is a 2 or a 4. Estimate by simulation the average number of throws Ford will make.

In this case, a simulation will not just be a single roll of the dice, but a sequence of rolls ending with a 2 or a 4. Note that in this case, it is not so easy to write down the probability of each outcome explicitly. This leads us to this (almost correct) solution:

```
import random, stdio

N = 10000
```

¹In fact, estimating probabilities is a special case of this more general scheme: what is the quantity you are calculating the average of in this case?

```

total = 0

for i in range(N):
    numrolls = 0

    die = 0
    while die != 2 and die != 4:
        die = 1 + int(random.random() * 6)
        numrolls += 1
    total += numrolls

avg = total / N
stdio.writeln(avg)

```

There is a slight bug in this solution, so that it will always yield an average of 1. Debug the problem so that you are getting valid simulation results. What answer do you obtain? Can you determine the correct theoretical answer?²

5 JWeb 1.3.20

Alice tosses a fair coin until she sees two consecutive heads. Bob tosses another fair coin until he sees a head followed by a tail.

Write a program `cointoss.py` to estimate the probability that Alice will make fewer tosses than Bob? Your program should take the number of simulations as its first command-line argument and produce output like the following:

```

$ python cointoss.py 3
Running 3 simulations.
0.6666666666666666

```

If your program is passed any other command-line arguments (regardless of what they are), you should produce output like the following describing what is happening in each simulation:

```

$ python cointoss.py 3 whateverlkewnfsohg
Running 3 simulations.
Alice:HTTHH Bob:HT
Alice:HH Bob:HHT Alice has fewer.
Alice:HH Bob:HT
0.3333333333333333

```

(Recall that the number of entries in `sys.argv` can be obtained using the `len()` function, so you can use that to test whether more than one argument has been passed.)

The exact answer is 39/121, so you can check your solution for many simulations — note that the examples above may give poor estimates, because the number of simulations is very low!

²For more information on this kind of situation, you may be interested in reading about the geometric distribution.

6 JBook 1.3.40

In the 1970s game show *Let's Make a Deal*, a contestant is presented with three doors. Behind one of them is a valuable prize. The contestant chooses a door, but does not open it. The host (who knows which door contains the prize) then opens one of the other two doors that does not contain the prize, showing the contestant that the prize is not there: note that there must be at least one such door, although there could be two. The contestant is then given the option to switch their choice to the other unopened door.

The question we consider here is: Should the contestant do so? Intuitively, it might seem that the contestant's initial choice and the other unopened door are equally likely to contain the prize, so there should be no incentive to switch.

Write a program `makeadeal.py` that takes a command-line argument N , simulates N scenarios as described above, and checks for each scenario which of the two strategies (switch or not switch) would lead to the contestant winning the prize. Use these simulations to estimate the chance of success for each strategy and print them out.³

Your program should take the number of simulations as its first parameter and produce output like the following:

```
$ python makeadeal.py 3
Running 3 simulations.
Probability of win if stays:0.3333333333333333
Probability of win if changes:0.6666666666666666
```

If your program is passed any other parameters (regardless of what they are), you should produce output like the following describing what is happening in each simulation:

```
$ python makeadeal.py 4 dsijfdsoihgsdl
Running 4 simulations.
winning_door=0 first_pick=1 opened_door=2 other_choice=0 Other choice wins!
winning_door=1 first_pick=0 opened_door=2 other_choice=1 Other choice wins!
winning_door=1 first_pick=1 opened_door=2 other_choice=0 First pick wins!
winning_door=2 first_pick=1 opened_door=0 other_choice=2 Other choice wins!
Probability of win if stays:0.25
Probability of win if changes:0.75
```

(Recall that the number of parameters passed to the program can be obtained from `len(sys.argv)`, so you can use that to test whether more than one parameter has been passed.)

Note that examples above may give poor estimates, because the number of simulations is very low!

7 JBook 1.4.10

Write a program `deal.py` which takes a command-line argument N , and prints N poker hands from a shuffled standard 52-card deck. Assume that we are playing *five-card stud* where a hand consists of 5 cards. You can assume $N \leq 10$.

Example usage:

³Additional comment: Since each of these two strategies choose different doors, and there are two unopened doors—with the prize behind one of them—one of the strategies will win each time. Therefore the true probabilities should add up to one, as should your estimated probabilities. As such, it is only actually necessary to simulate one of the strategies to answer this question.

```
$ python deal.py 2
4 of Spades
9 of Spades
Ace of Hearts
9 of Clubs
9 of Diamonds

6 of Spades
10 of Hearts
Queen of Hearts
8 of Hearts
King of Spades
```

8 JBook 1.4.34 [Challenging]

In the game of bridge, four players are dealt hands of 13 cards each from a standard 52-card deck. An important property of a bridge hand is the distribution of the number of cards of each suit in the hand.

Write a program `bridgehands.py` to test which of the following distributions is the most likely: 5-3-3-2, 4-4-3-2, or 4-3-3-3. (Here the numbers represent the number of cards of each suit in the hand, ordered from longest to shortest suit.)

Your program must take an integer N as a command-line argument, where N is the number of times the players must be dealt a hand (i.e. how many games they play). Note that this corresponds to $4N$ hands that should be analyzed when estimating the probabilities. Your program should print the probability of observing each of the three distributions mentioned, and which is then the most likely distribution.

Example usage—if more than one of the distributions are equally most likely, your output may print any one as the most likely:

```
$ python bridgehands.py 3
Probability of 5-3-3-2: 0.16666667
Probability of 4-4-3-2: 0.083333336
Probability of 4-3-3-3: 0.083333336
5-3-3-2 is the most likely.
```

Note that the estimates in the example above may be inaccurate, due to the small number of simulations.

Hint: You can use the built-in function `sorted()` to obtain a sorted version of a list, such as a list of the number of cards in each suit.